

WEEK 3

SELLER AND INVENTORY MANAGEMENT SYSTEM

Database Activity Report using Oracle SQL

Module	Seller and Inventory Management
Database	Oracle SQL
Main Tables	SELLER, INVENTORY
Key Concepts	Primary Key, Foreign Key, UNIQUE, NOT NULL, CHECK
Reports	Seller-wise Product, Stock Available, Inventory Analysis

Scenario: The system maintains seller details and tracks product inventory, stock quantity and stock status. Relationships and constraints support data accuracy and consistency.

1. Create Seller and Inventory Tables

The SELLER table stores seller ID, name, email, phone and address. The INVENTORY table stores inventory ID, product ID, seller ID, stock quantity, stock status and last update date.

Seller Table

```
SQL> CREATE TABLE seller(seller_id varchar(7),seller_name varchar(25),email varchar(30),phone number(10),address varchar(45));
```

Table created.

Inventory Table

Table created.

```
SQL> CREATE TABLE inventory_id varchar(7),product_id varchar(7),seller_id varchar(7),stock_quantity number,stock_status varchar(11),last_update DATE);
CREATE TABLE inventory_id varchar(7),product_id varchar(7),seller_id varchar(7),stock_quantity number,stock_status varchar(11),last_update DATE)
```

2. Establish Relationship

A seller can supply one or more products. SELLER_ID identifies the seller, PRODUCT_ID identifies the product, and INVENTORY connects the related records.

Relationship: SELLER → INVENTORY ← PRODUCT

3. Primary Key Added to Both Tables

A primary key uniquely identifies each row and prevents duplicate identifiers.

```
SQL> SELECT * FROM seller;

SELLER_ SELLER_NAME      EMAIL                                P
-----
ADDRESS
S00001  vani enterprises        mdvanienter@gmail.com          876543
2109
anna nagar, chennai

S00002  sd & co                 sellerd827@gmail.com           98767
8544
62,thirumalai nagar,thanjavur

S00003  a to z                  atoz@gmail.com                 78967
8545
90, george town,chennai

SELLER_ SELLER_NAME      EMAIL                                P
-----
ADDRESS
S00004  DSVV manufaturers      dsvvmanuf142@gmail.com        78545
6789
23,thillai nagar ,trichy
```

4. Foreign Key for INVENTORY Table

INVENTORY.PRODUCT_ID references PRODUCT.PRODUCT_ID and INVENTORY.SELLER_ID references SELLER.SELLER_ID. Foreign keys maintain valid relationships between tables.

```
SQL> INSERT INTO seller VALUES('&sellerllid','&sellername','&email','&phone','&address');
Enter value for sellerllid: S00001
Enter value for sellername: vani enterprises
Enter value for email: mdvanienter@gmail.com
Enter value for phone: 8765432109
Enter value for address: anna nagar, chennai
old 1: INSERT INTO seller VALUES('&sellerllid','&sellername','&email','&phone','&address')
new 1: INSERT INTO seller VALUES('S00001','vani enterprises','mdvanienter@gmail.com','8765432109','anna nagar, chennai')

1 row created.
```

5. Constraint to Make Email ID Unique

The UNIQUE constraint prevents duplicate seller email addresses.

```
SQL> ALTER TABLE inventory ADD CONSTRAINT fk_prod FOREIGN KEY(product_id) REFERENCES product(product_id);
Table altered.

SQL> ALTER TABLE inventory ADD CONSTRAINT fk_seller FOREIGN KEY(seller_id) REFERENCES seller(seller_id);
Table altered.
```

6. Make Email NOT NULL

The NOT NULL constraint requires every seller record to contain an email address.

```
SQL> ALTER TABLE seller ADD CONSTRAINT un_mail UNIQUE(email);
Table altered.
```

7. Check Stock Quantity is Positive

The CHECK constraint allows stock quantity values greater than or equal to zero.

```
SQL> ALTER TABLE seller MODIFY email NOT NULL;
Table altered.
```

8. Insert Values into SELLER Table

INSERT adds a new seller record using seller ID, name, email, phone and address. The supplied output confirms that one row was created.

```
SQL> ALTER TABLE inventory ADD CONSTRAINT ck_quan CHECK(stock_quantity >=0);
Table altered.
```

9. Insert / View Inventory Values

Inventory records associate products with sellers and store current stock quantity and stock status.

```
SQL> SELECT s.seller_name, COUNT(i.product_id) AS number_of_products, SUM(i.
stock_quantity) AS Available_stock FROM seller s JOIN inventory i ON s.selle
r_id = i.seller_id WHERE i.stock_status = 'Available' GROUP BY s.seller_name
;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
ABC Enterprises	1	3
DEF Enterprises	1	3
vani enterprises	1	4

```
SQL> SELECT p.product_name, i.stock_quantity, i.stock_status FROM product p
JOIN inventory i ON p.product_id = i.product_id;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATU
Laptop	4	Available
Laptop	3	Available
Mobile Phone	5	Unavailable

```
PRODUCT_NAME
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATU
Keyboard	3	Available

Report 1 – Seller-wise Product Report

This report groups available inventory by seller and shows the number of available products and total available stock for each seller.

```
SQL> SELECT s.seller_name, COUNT(i.product_id) AS number_of_products, SUM(i.
stock_quantity) AS Available_stock FROM seller s JOIN inventory i ON s.selle
r_id = i.seller_id WHERE i.stock_status = 'Available' GROUP BY s.seller_name
;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
ABC Enterprises	1	3
DEF Enterprises	1	3
vani enterprises	1	4

Report 2 – Stock Available Report

This report counts inventory records whose stock status is Available.

```
SQL> SELECT COUNT(*) AS total_Available_products FROM inventory WHERE stock_
status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
3

Finding: Total available products = 3.

Report 3 – Inventory Analysis

Inventory analysis uses COUNT, MAX and AVG to summarize stock levels.

Total Product Available

```
SQL> SELECT COUNT(*) AS total_Available_products FROM inventory WHERE stock_
status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
3

Result: 3 available products.

Product Out of Stock

The out-of-stock query counts records whose status is out of stock.

```
SQL> SELECT COUNT(*) AS outofstock_products FROM inventory WHERE stock_status = 'out of stock';

OUTOFSTOCK_PRODUCTS
-----
0
```

Result: 0 out-of-stock products.

Highest Stocked Product

```
SQL> SELECT p.product_name,i.stock_quantity FROM product p JOIN inventory i
ON p.product_id = i.product_id WHERE i.stock_quantity = (SELECT MAX (stock_quantity) FROM inventory);

PRODUCT_NAME
-----
STOCK_QUANTITY
-----
Mobile Phone      5
```

Result: Mobile Phone – stock quantity 5.

Average Inventory Quantity

```
SQL> SELECT AVG(stock_quantity) AS average_inventory FROM inventory;

AVERAGE_INVENTORY
-----
3.75
```

Result: Average inventory quantity = 3.75.

Conclusion

The Week 3 activity demonstrates Oracle SQL table creation, relationships, primary and foreign keys, UNIQUE, NOT NULL and CHECK constraints, record insertion, and inventory reporting.

Summary of supplied results

Analysis	Result
Total product available	3
Product out of stock	0
Highest stocked product	Mobile Phone – 5
Average inventory quantity	3.75